
Robot Arm 1

— Electric, Electronic and System
Engineering —

Homogenous Transformation

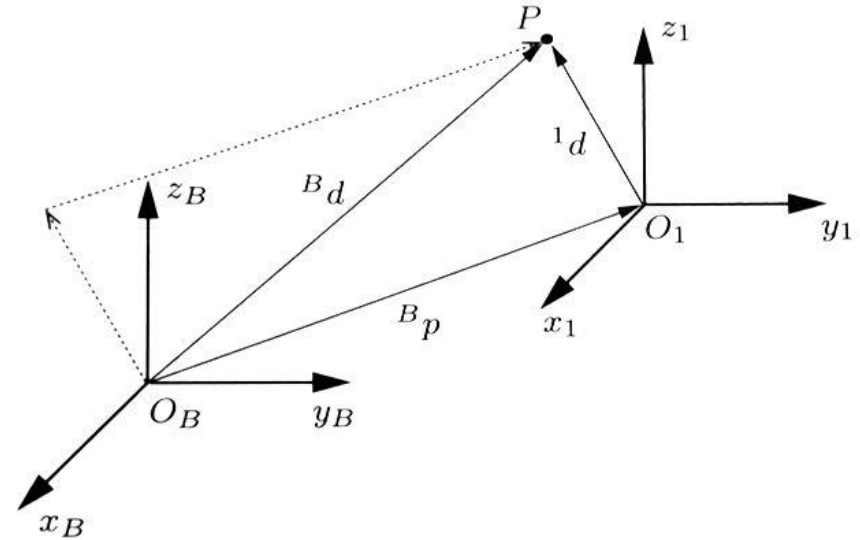
Translational Movement

Coordinate system O_1 is made with translational movement p from O_B .

$${}^B\mathbf{p} = p_x\mathbf{i} + p_y\mathbf{j} + p_z\mathbf{k}$$

where, $\mathbf{i}, \mathbf{j}, \mathbf{k}$

is unit vector that direction is x, y, z .



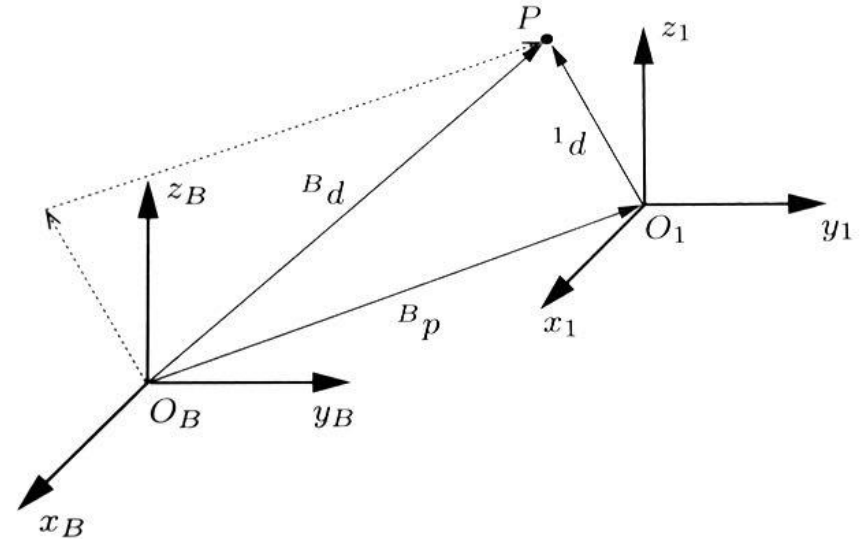
Translational Movement

${}^B\mathbf{d}$ Point P on coordinate O_B .

${}^1\mathbf{d}$ Point P on coordinate O_1 .

$${}^B\mathbf{d} = {}^1\mathbf{d} + \text{?}$$

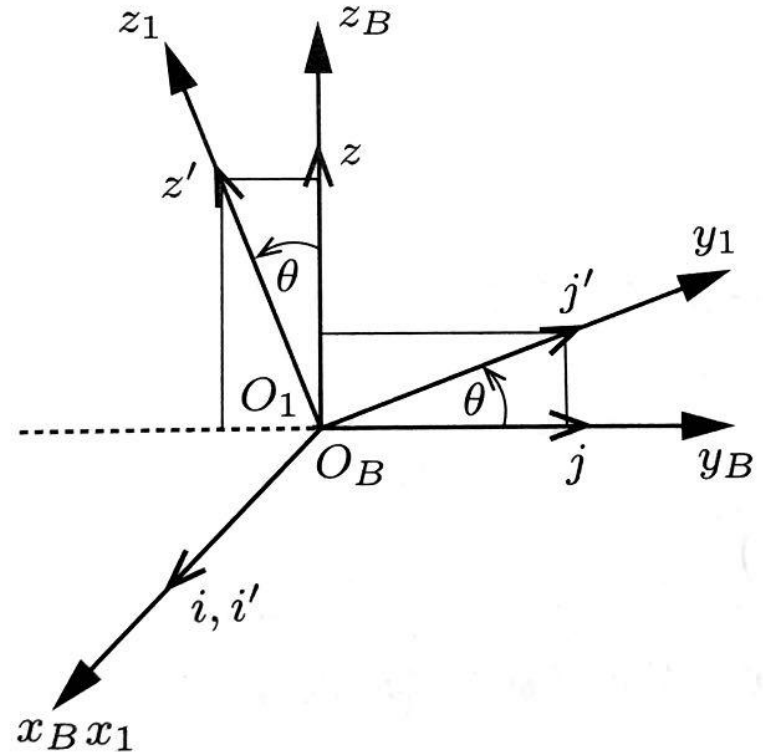
Translational movement is described with sum of vectors.



Rotational Movement

Coordinate system O_1 is made with rotation θ around x axis on O_B .

$$[\mathbf{i}' \quad \mathbf{j}' \quad \mathbf{k}'] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \text{cloud} & 0 \\ 0 & \text{cloud} & ? \end{bmatrix}$$

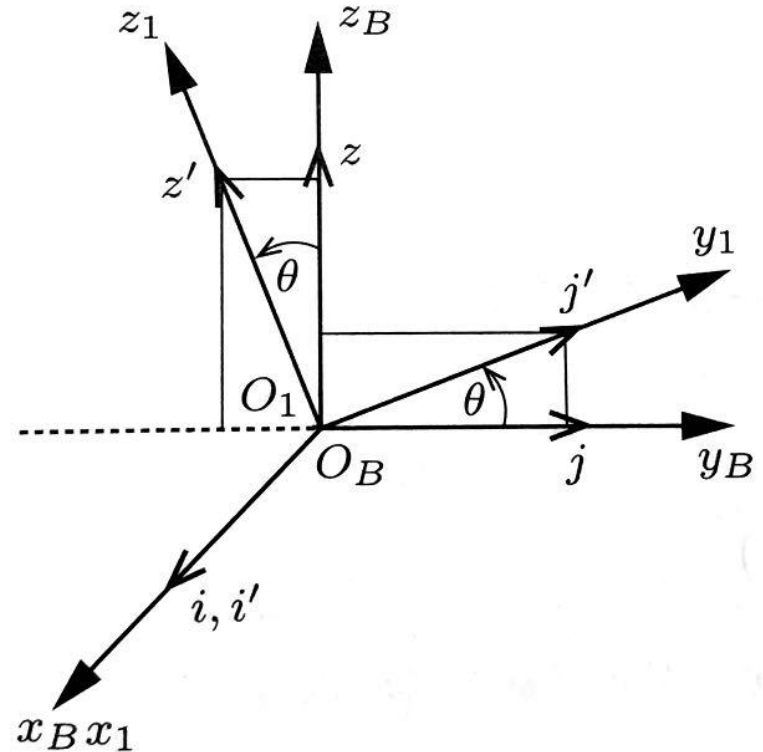


Rotational Movement

Here, we introduce rotation matrix R.

$$[i' \ j' \ k'] = {}^B R_1(x, \theta)[i \ j \ k]$$

$${}^B R_1(x, \theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$



Rotational Movement

Therefore, matrix R around y axis is

$${}^B\mathbf{R}_1(y, \theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

matrix R around z axis is

$${}^B\mathbf{R}_1(z, \theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

respectively.

Combination of Roll, Pitch, Yaw Rotation

In the case of Robot Arm with Roll, Pitch and Yaw axes, whole rotation matrix is

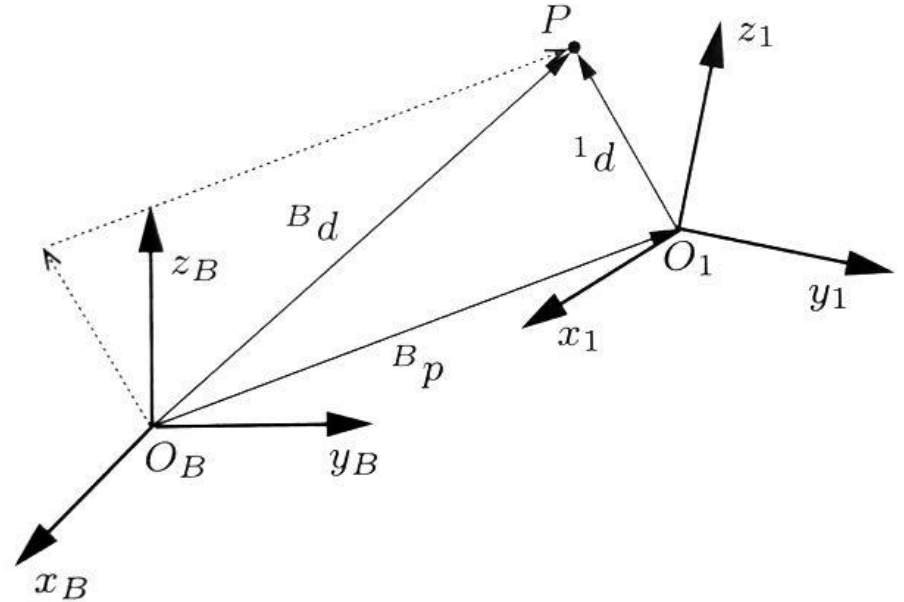
$$\begin{aligned} {}^B\mathbf{R}_1 &= R(z, \phi) R(y', \theta) R(x'', \psi) \\ &= \begin{bmatrix} C_\phi C_\theta & C_\phi S_\theta S_\psi - S_\phi C_\psi & C_\phi S_\theta C_\psi + S_\phi S_\psi \\ S_\phi C_\theta & S_\phi S_\theta S_\psi + C_\phi C_\psi & S_\phi S_\theta C_\psi - C_\phi C_\psi \\ -S_\theta & C_\theta S_\psi & C_\theta C_\psi \end{bmatrix} \end{aligned}$$

where, $S_* \equiv \sin *$, $C_* \equiv \cos *$

Homogenous transformation

Coordinate system O_1 is made with translational movement p and rotational movement R from O_B .

$${}^B\mathbf{d} = {}^B R_1 {}^1\mathbf{d} + \text{?}$$



Homogenous transformation

For simplicity, we introduce new vector d' .

$${}^B d' = \begin{bmatrix} {}^B d \\ \mathbf{1} \end{bmatrix}, {}^1 d' = \begin{bmatrix} {}^1 d \\ \mathbf{1} \end{bmatrix}$$

Then, we got simple equation to transformation from B to 1 as;

$$\begin{bmatrix} {}^B d \\ \mathbf{1} \end{bmatrix} = \begin{bmatrix} {}^B R_1 & {}^B p \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} {}^1 d \\ \mathbf{1} \end{bmatrix}$$

$${}^B d' = A^1 d'$$

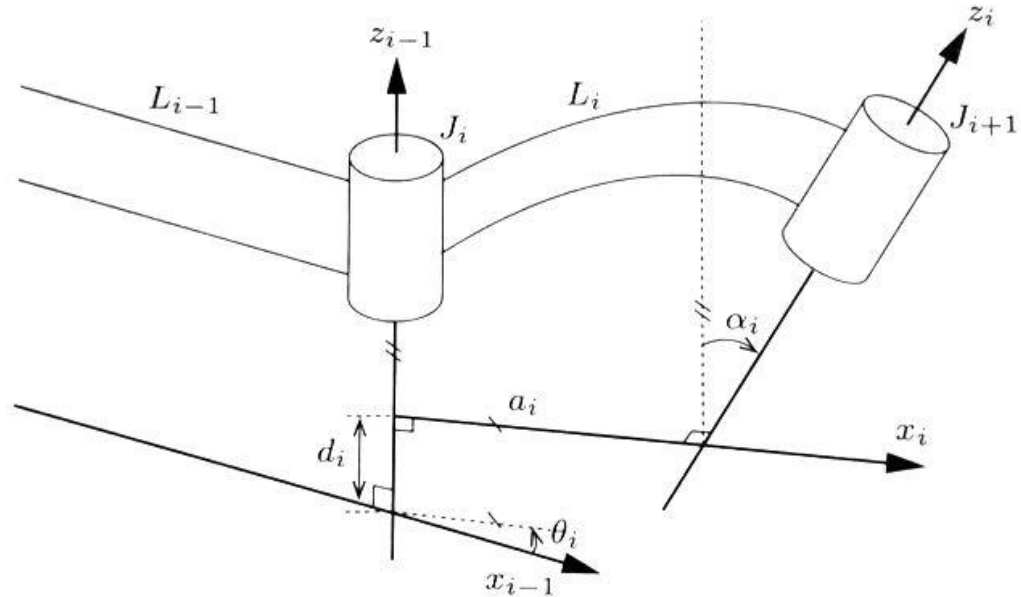
Kinematics & Inverse Kinematics

Denavit-Hartenberg method

d_i : distance from origin of coordinate(i-1) to crosspoint of $z(i-1)$ axis and $x(i)$ axis.

a_i : distance from above crosspoint to origin of coordinate(i)

θ_i : rotation angle from $x(i-1)$ axis to $x(i)$ axis around $z(i-1)$ axis.

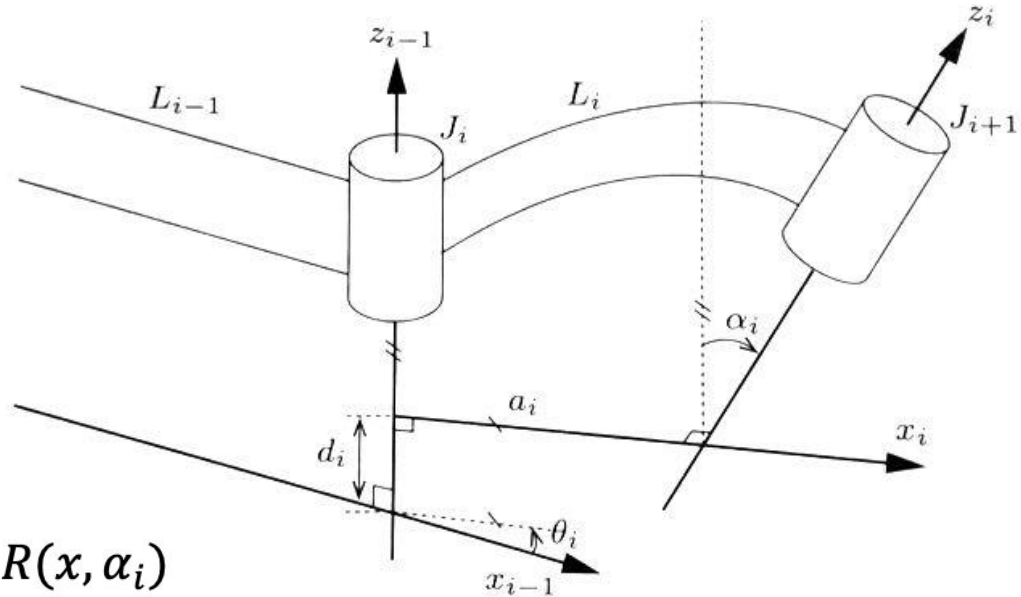


Denavit-Hartenberg method

α_i : rotation angle from $z_{(i-1)}$ axis to z_i axis around x_i axis.

Therefore, homogenous transformation from coordinate $i-1$ to coordinate i is described as;

$${}^{i-1}\mathbf{A}_i = R(z, \theta_i) T(0,0, d_i) T(a_i, 0,0) R(x, \alpha_i)$$



Kinematics

where, T is translational movement matrix as;

$$T = \begin{bmatrix} 1 & 0 & 0 & \\ 0 & 1 & 0 & \mathbf{p} \\ 0 & 0 & 1 & \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, we got transformation matrix as;

$${}^{i-1}\mathbf{A}_i = \begin{bmatrix} C_{\theta_i} & -S_{\theta_i}C_{\alpha_i} & S_{\theta_i}S_{\alpha_i} & a_iC_{\theta_i} \\ S_{\theta_i} & C_{\theta_i}C_{\alpha_i} & -C_{\theta_i}S_{\alpha_i} & a_iS_{\theta_i} \\ 0 & S_{\alpha_i} & C_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

N Link Manipulator

In case of N link manipulator, homogenous transformation matrix becomes;

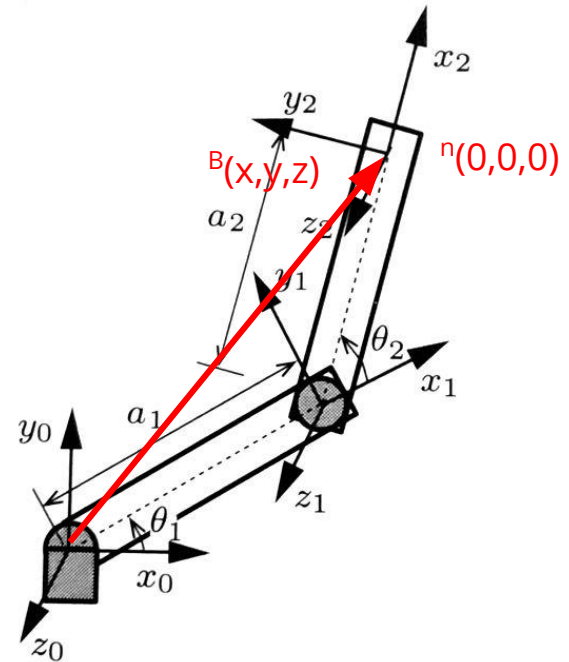
$${}^B\mathbf{A}_n = {}^B\mathbf{A}_1 {}^1\mathbf{A}_2 {}^2\mathbf{A}_3 \cdots {}^{n-1}\mathbf{A}_n$$

ex. tip position of Nth-link

in base coordinate is described as;

$${}^B\mathbf{d}' = {}^B\mathbf{A}_n {}^n\mathbf{d}'$$

$$\begin{bmatrix} {}^Bx \\ {}^By \\ {}^Bz \\ 1 \end{bmatrix} = {}^B\mathbf{A}_n \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

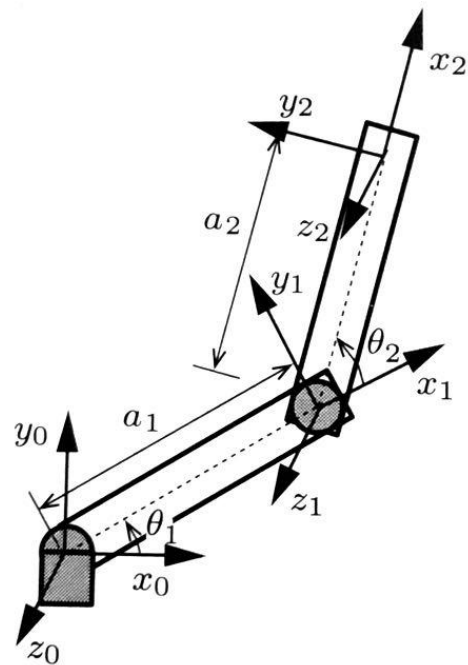


2-Link Manipulator

Kinematics

$${}^B\mathbf{A}_1 = \begin{bmatrix} C_{\theta_1} & -S_{\theta_1} & 0 & a_1 C_{\theta_1} \\ S_{\theta_1} & C_{\theta_1} & 0 & a_1 S_{\theta_1} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^1\mathbf{A}_2 = \begin{bmatrix} C_{\theta_2} & -S_{\theta_2} & 0 & a_2 C_{\theta_2} \\ S_{\theta_2} & C_{\theta_2} & 0 & a_2 S_{\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

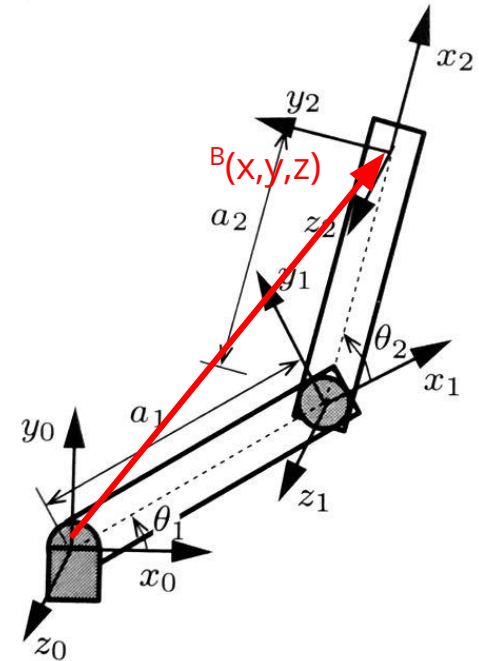


Kinematics

$${}^B\mathbf{A}_2 = \begin{bmatrix} C_{\theta_1+\theta_2} & -S_{\theta_1+\theta_2} & 0 & a_2 \\ S_{\theta_1+\theta_2} & C_{\theta_1+\theta_2} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From the following equation, tip position is described;

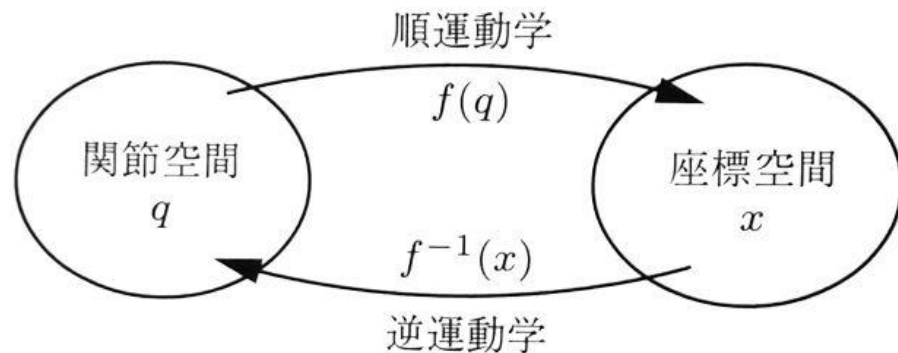
$$\begin{bmatrix} {}^Bx \\ {}^By \\ {}^Bz \\ 1 \end{bmatrix} = {}^B\mathbf{A}_n \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} c_{\theta_1+\theta_2} a_2 \\ s_{\theta_1+\theta_2} a_2 \\ 0 \\ 1 \end{bmatrix}$$



Kinematics & Inverse Kinematics

Tip position can be solved from joint angle using kinematics.

On the other hand, joint angle **may be solved** from tip position using inverse kinematics.



$$x = f(q), \quad q = f^{-1}(x)$$

Inverse Kinematics on N link Manipulator

$${}^B A_n = {}^B A_1 {}^1 A_2 {}^2 A_3 \cdots {}^{n-1} A_n$$

$${}^B A_n = {}^B A_1 {}^1 A_n$$

$${}^B A_1^{-1} {}^B A_n = {}^1 A_n$$

Base -> Nth link

=Base -> 1st link -> Nth link

1st -> Base -> Nth = 1st -> Nth

Ex. 2 Link Manipulator

$${}^B\mathbf{A}_1^{-1}{}^B\mathbf{A}_2 = {}^1\mathbf{A}_2 \qquad {}^B\mathbf{A}_2 = \begin{bmatrix} C_{\theta_1+\theta_2} & -S_{\theta_1+\theta_2} & 0 & a_1C_{\theta_1} + a_2C_{\theta_1+\theta_2} \\ S_{\theta_1+\theta_2} & C_{\theta_1+\theta_2} & 0 & a_1S_{\theta_1} + a_2S_{\theta_1+\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^B\mathbf{A}_1^{-1} = \begin{bmatrix} C_{\theta_1} & S_{\theta_1} & 0 & -a_1 \\ -S_{\theta_1} & C_{\theta_1} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

If the position of manipulator tip (x, y)
is known,

$${}^1\mathbf{A}_2 = \begin{bmatrix} C_{\theta_2} & -S_{\theta_2} & 0 & a_2C_{\theta_2} \\ S_{\theta_2} & C_{\theta_2} & 0 & a_2S_{\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^B\mathbf{A}_2 = \begin{bmatrix} C_{\theta_1+\theta_2} & -S_{\theta_1+\theta_2} & 0 & x \\ S_{\theta_1+\theta_2} & C_{\theta_1+\theta_2} & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ex. 2 Link Manipulator

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_1 C_{\theta_1} + a_2 C_{\theta_1+\theta_2} \\ a_1 S_{\theta_1} + a_2 S_{\theta_1+\theta_2} \end{bmatrix}$$

← kinematics information

↓ additional information

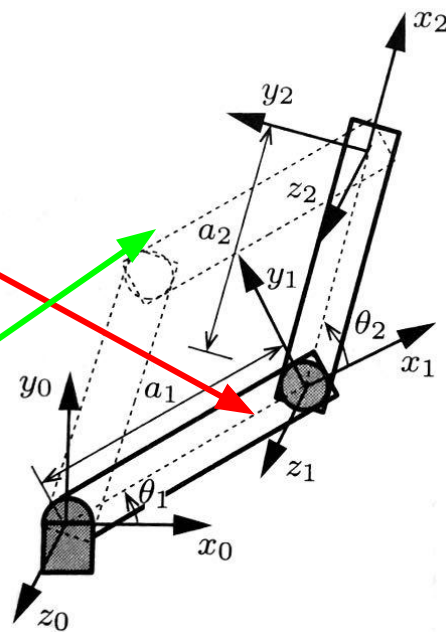
$$\begin{bmatrix} C_{\theta_1} & S_{\theta_1} & 0 & -a_1 \\ -S_{\theta_1} & C_{\theta_1} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} C_{\theta_1+\theta_2} & -S_{\theta_1+\theta_2} & 0 & x \\ S_{\theta_1+\theta_2} & C_{\theta_1+\theta_2} & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} C_{\theta_2} & -S_{\theta_2} & 0 & a_2 C_{\theta_2} \\ S_{\theta_2} & C_{\theta_2} & 0 & a_2 S_{\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ex. 2 Link Manipulator

Finally, we get,

$$\begin{cases} \theta_1 = \text{atan2} \left(\frac{x^2 + y^2 + a_1^2 - a_2^2}{2a_1\sqrt{x^2 + y^2}}, \sqrt{1 - \left(\frac{x^2 + y^2 + a_1^2 - a_2^2}{2a_1\sqrt{x^2 + y^2}} \right)^2} \right) \\ \theta_2 = \text{atan2} \left(\sqrt{1 - \left(\frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2} \right)^2}, \frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2} \right) \end{cases}$$

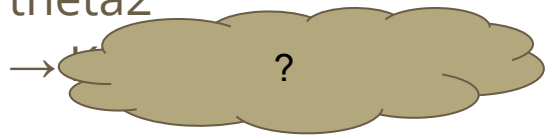
$$\begin{cases} \theta_1 = \text{atan2} \left(\frac{x^2 + y^2 + a_1^2 - a_2^2}{2a_1\sqrt{x^2 + y^2}}, -\sqrt{1 - \left(\frac{x^2 + y^2 + a_1^2 - a_2^2}{2a_1\sqrt{x^2 + y^2}} \right)^2} \right) \\ \theta_2 = \text{atan2} \left(-\sqrt{1 - \left(\frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2} \right)^2}, \frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2} \right) \end{cases}$$



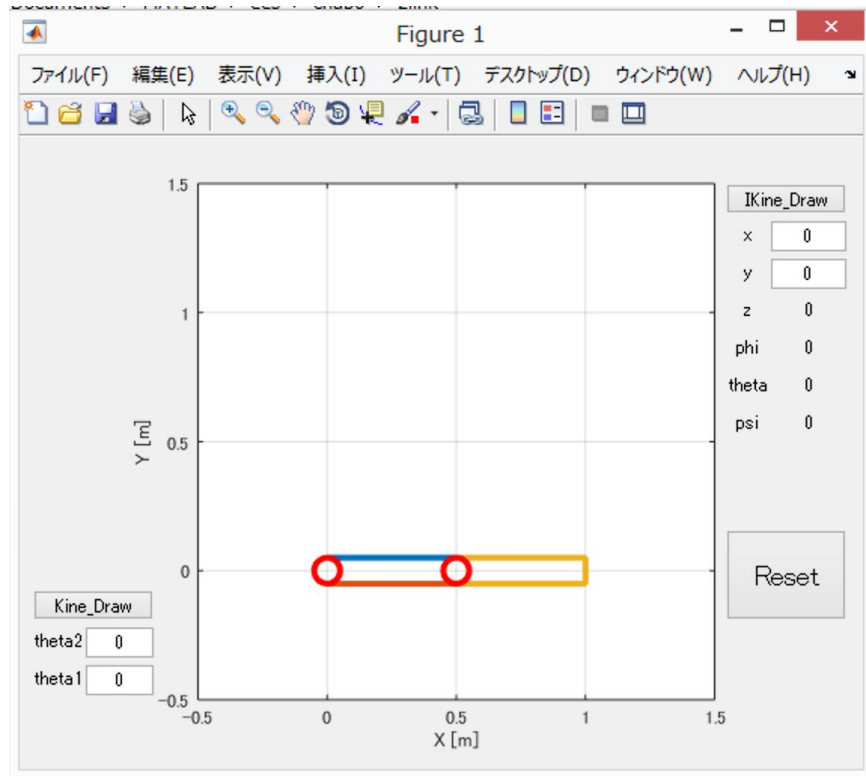
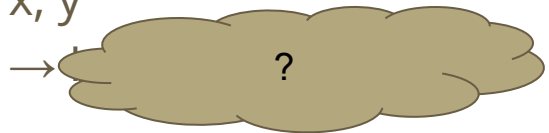
Draw Robots with MATLAB

There are 2 functions

1. Draw robot with θ_1 , θ_2



2. Draw robot with tip position x, y



About DrawRobot.m

Homogenous Transform matrices are written in line 33 to 45.

Position of manipulator base and link1 and link2 and tip are calculated inline 46 to 55.

```
33 - A0=[cos(theta1),-sin(theta1),0,0;...
34     sin(theta1),cos(theta1),0,0;...
35     0,0,1,0;...
36     0,0,0,1]; %座標系 1から基準座標系への同次変換行列
37 - A1=[cos(theta1),-sin(theta1),0,a1*cos(theta1);...
38     sin(theta1),cos(theta1),0,a1*sin(theta1);...
39     0,0,1,0;...
40     0,0,0,1]; %座標系 2から座標系 1への同次変換行列
41 - A2=[cos(theta2),-sin(theta2),0,a2*cos(theta2);...
42     sin(theta2),cos(theta2),0,a2*sin(theta2);...
43     0,0,1,0;...
44     0,0,0,1]; %先端座標系から座標系 2への同次変換行列
45 %アーム位置の更新
46 - npB0_1=A0*pB0_1;
47 - npB0_2=A0*pB0_2;
48 - npB1=A1*pB0;
49 - npB1_1=A1*pB0_1;
50 - npB1_2=A1*pB0_2;
51 - npB2_1=A1*[A2(1:3,1:3),zeros(3,1);zeros(1,3),1]*pB0_1;
52 - npB2_2=A1*[A2(1:3,1:3),zeros(3,1);zeros(1,3),1]*pB0_2;
53 - npT=A1*A2*pB0;
54 - npT_1=A1*A2*pB0_1;
55 - npT_2=A1*A2*pB0_2;
```

About DrawRobot.m

Inverse Kinematics can not be solved simply. All thetas are calculated according to the previous slide.

Line 9 to 22

→a solution of inverse kinematics

Line 23 to 32

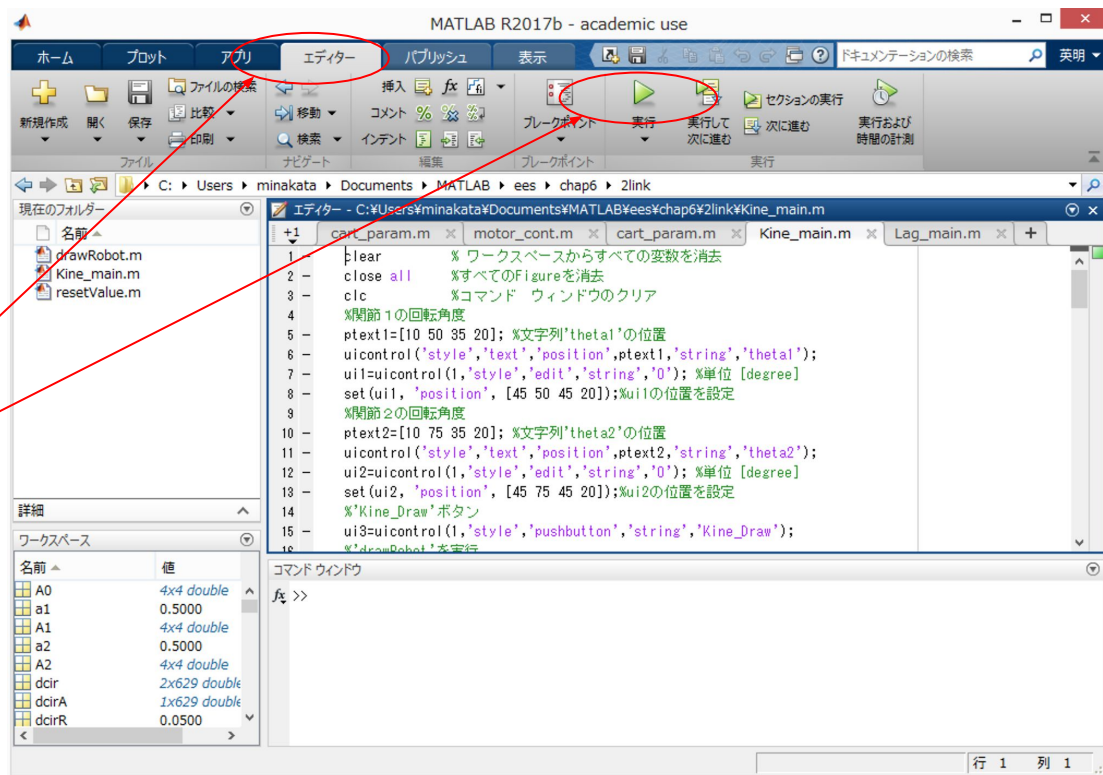
→another solution of inverse kinematics

```
8 - end
9 - if flag==1 %逆運動学 ケース 1
10 -     %先端位置xの値をui6から読み込み
11 -     x=str2double(get(ui6,'string'));
12 -     %先端位置yの値をui7から読み込み
13 -     y=str2double(get(ui7,'string'));
14 -     %関節1の回転角度を計算
15 -     theta1=atan2((x^2+y^2+a1^2-a2^2)/(2*a1*sqrt(x^2+y^2)),...
16 -         sqrt(1-((x^2+y^2+a1^2-a2^2)/(2*a1*sqrt(x^2+y^2)))^2))...
17 -         -atan2(x,y);
18 -     %関節2の回転角度を計算
19 -     theta2=atan2(sqrt(1-((x^2+y^2-a1^2-a2^2)/(2*a1*a2))^2),...
20 -         (x^2+y^2-a1^2-a2^2)/(2*a1*a2));
21 -     set(ui12,'Visible','on'); %ui12 ('Next'ボタン) を可視
22 - end
23 - if flag==2 %逆運動学 ケース 2
24 -     x=str2double(get(ui6,'string'));
25 -     y=str2double(get(ui7,'string'));
26 -     theta1=atan2((x^2+y^2+a1^2-a2^2)/(2*a1*sqrt(x^2+y^2)),...
27 -         -sqrt(1-((x^2+y^2+a1^2-a2^2)/(2*a1*sqrt(x^2+y^2)))^2))...
28 -         -atan2(x,y);
29 -     theta2=atan2(-sqrt(1-((x^2+y^2-a1^2-a2^2)/(2*a1*a2))^2),...
30 -         (x^2+y^2-a1^2-a2^2)/(2*a1*a2));
```

Kine_main.m Controls this Drawing

Kine_main.m controls drawing of the robot

1. open 'Kine_main.m'
2. move to 'editor' tab
3. click 'run'



Ex. 1 Kinematics Drawing

Input

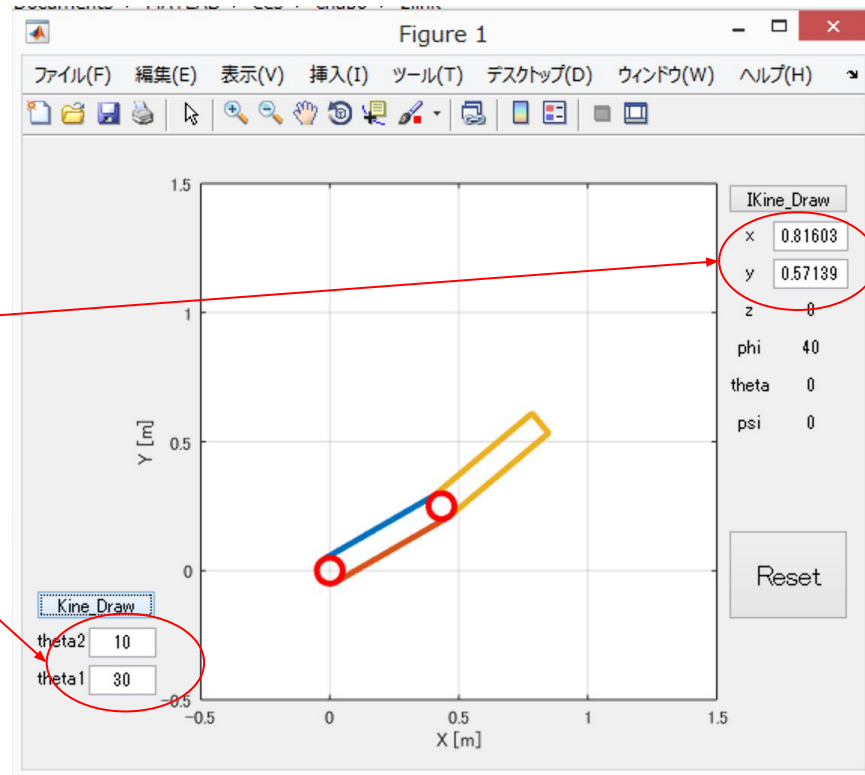
theta1: 30 deg

theta2: 10 deg

Output

x: 0.81603

y: 0.57139



Ex.2 Inverse Kinematics Drawing

Input

x: 0.8

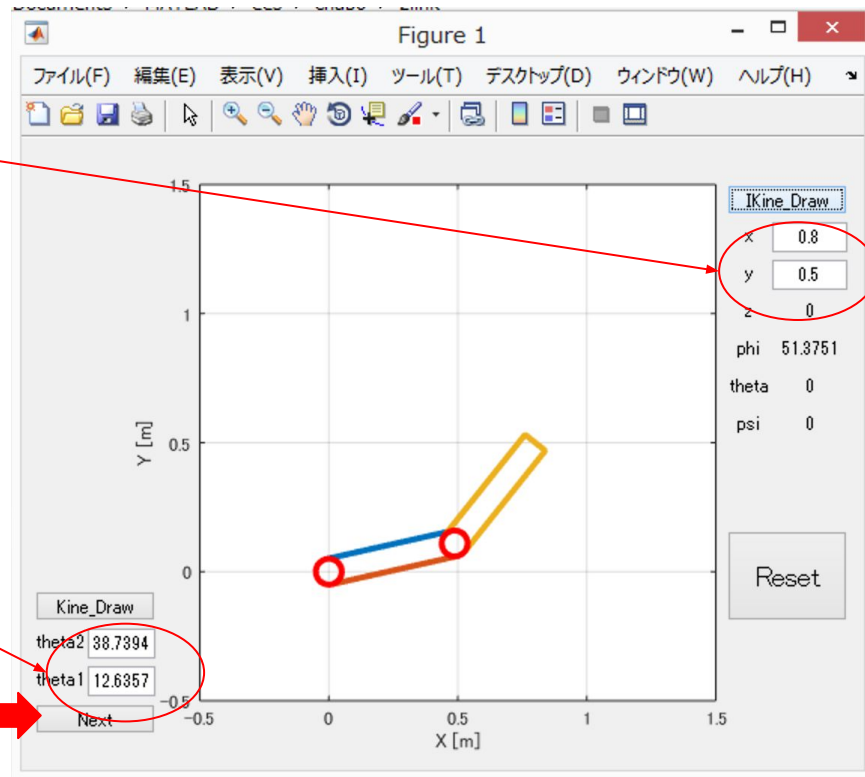
y: 0.5

Output

theta1: 12.6357 deg

theta2: 38.7394 deg

There's a "Next" button here!



Ex.2-1 Inverse Kinematics Drawing

Input

x: 0.8

y: 0.5

We got another Output

theta1: 51.3751 deg

theta2: -38.7394 deg

